

Introduction à la programmation graphique avec pygame

Équipe Développement Efficace

12 septembre 2025

1 Découverte de pygame

Au cours de ce module d'algorithmique, nous allons utiliser la bibliothèque `pygame` pour réaliser des interfaces graphiques. Dans ce TP, nous allons nous initier à son maniement.

La page web de `pygame` se trouve à <http://www.pygame.org>. `pygame` fournit un *canevas*, c'est à dire une fenêtre dans laquelle on peut dessiner des formes (rectangles, ellipses, droites, courbes), et un moyen de récupérer les clics et autres actions de l'utilisateur. Dans `pygame`, le contenu du canevas n'a pas de structure, contrairement à ce qui se passe en html, mais uniquement des pixels sur lesquels on peut opérer.

Créer une application `pygame` demande quelques étapes ; on vous fournit le fichier `squelette.py` qui contient une application minimale.

Question 1 *Échauffement*

Lancer l'application `squelette` avec la commande `python3 squelette.py`.

Question 2 *Les goûts et les couleurs*

Changer la couleur de la balle. Indication : ce magnifique cyan est la couleur rgb 123,234,222.

Question 3 *Une question de taille*

Changer la taille de la balle.

Question 4

Que représentent les paramètres de la fonction `pygame.draw.circle` ? Vérifier dans votre réponse dans la documentation.

Question 5 *Revoyons l'action au ralenti*

On veut voir la balle se déplacer étape par étape. Pour cela, il faut afficher l'animation à une vitesse de 2 images par secondes (2 FPS) au lieu de 30. Effectuer la modification nécessaire.

Quelle est l'instruction qui provoque l'attente entre deux images ?

Passer l'animation en 60 FPS pour plus un confort de visionnage incomparable.

Question 6 *Un petit rafraîchissement*

À quoi sert la fonction `refresh` ? Que se passe-t-il si on ne l'appelle pas ?

2 Un micro-jeu

2.1 La base

Puisque la bibliothèque s'appelle **pygame**, nous allons créer un micro-jeu à partir de `squelette.py`.

Commençons par entraîner le joueur à cliquer sur la balle. Pour cela, il faut d'abord savoir où se trouve la balle, donc avoir un objet qui la représente.

Question 7 *Une question de classe*

Dans un nouveau fichier `balle.py`, créer une classe `Balle`, avec des attributs `position_x`, `position_y`, `vitesse_x`, `vitesse_y`, `couleur` et `taille`.

Quels sont les types de ces attributs ? Pour les attributs numériques, en quelles *unités* sont-ils exprimés ?

Question 8 *On avance*

Ajouter à la classe `Balle` une méthode `avance(t)` qui modifie la position de la balle pour refléter le fait que “t” millisecondes se sont écoulées.

Question 9 *Méthode de dessin*

Ajouter à la classe `Balle` une méthode `dessine(s)` qui dessine la balle sur le canevas `pygame s` passé en argument.

Question 10 *La question à deux balles*

Utiliser la classe `Balle` depuis `squelette.py` pour créer une application avec deux balles qui se déplacent (avec des vitesses différentes).

Question 11 *Dans le mille*

Ajouter à la classe `Balle` une méthode `contient(position)` qui indique si la position passée en argument est située à l'intérieur de la balle.

Question 12 *La récompense*

Le score du joueur est le nombre de fois où il ou elle a cliqué sur l'une des balles. Ajouter le score dans l'application et l'afficher.

2.2 Les suppléments

Question 13 *Niveau 2*

Ajouter une nouvelle balle (avec une vitesse et une couleur aléatoires) à chaque fois que le joueur clique sur une balle existante.

Question 14 *Niveau hardcore*

Ajouter une balle (et augmenter le score) uniquement si le joueur clique sur chaque balle sans cliquer deux fois sur la même. Quand le joueur re-clique sur une balle qui a déjà été cliquée, on oublie quelles balles ont été cliquées précédemment.

Question 15 *Moteur physique dernière génération*

Que peut-on modifier pour que les balles rebondissent sur les bords de l'écran ?
Indication : on peut commencer par se contenter de les faire rebondir quand leur *centre* atteint le bord de l'écran.